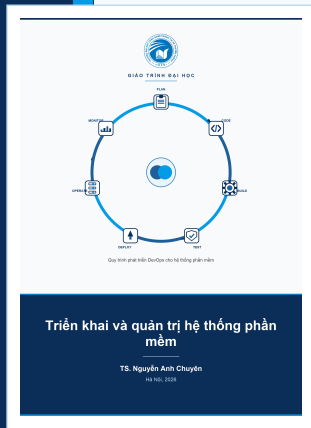




CHƯƠNG 8

Quản lý Log & Cảnh báo sự cố

Triển khai và quản trị hệ thống phần mềm



Giới thiệu nội dung

Mục tiêu chương

- Giải thích vai trò quản lý log tập trung và so sánh các giải pháp phổ biến (ELK, Splunk, Grafana Loki).
- Triển khai Loki và Promtail bằng Docker Compose để thu thập log từ WordPress và MySQL.
- Viết truy vấn LogQL; cấu hình Alertmanager và alerting rules để cảnh báo sự cố.

Nội dung chính

- 1** Vai trò của log và giải pháp quản lý tập trung
- 2** Kiến trúc và triển khai Grafana Loki
- 3** Truy vấn LogQL và trực quan hóa
- 4** Alertmanager và cảnh báo sự cố

Log: “hộp đen” của hệ thống CNTT

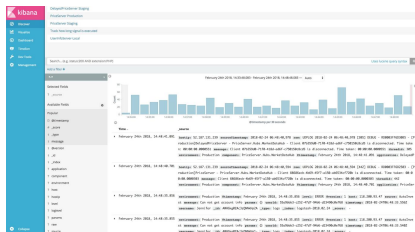
- **Log (nhật ký)** ghi lại toàn bộ hoạt động, sự kiện và lỗi xảy ra trên ứng dụng hoặc hạ tầng.
- **Log Server** thu thập, lưu trữ, quản lý và phân tích log từ nhiều nguồn: server, application, database, network device, cloud service.
- Thay vì lưu log phân tán trên từng máy, **centralized logging** tập trung toàn bộ log về một nơi để hỗ trợ monitoring, troubleshooting và security analysis.
- Với hàng trăm/ngày service chạy đồng thời, kiểm tra log thủ công từng máy chủ là bất khả thi.

Câu hỏi mở đầu

Chương 7 đã triển khai Prometheus + Grafana để giám sát **metrics**. Nếu ứng dụng lỗi lúc 3 giờ sáng, chỉ số CPU/RAM có đủ để biết **nguyên nhân** là gì không? Đó là lý do cần thêm **log**.

Vì sao cần thu thập log tập trung?

- **Giám sát:** theo dõi trạng thái server, application, network thời gian thực để phát hiện sớm bất thường.
- **Xử lý sự cố:** truy vết nguyên nhân gốc rễ nhanh hơn, giảm chỉ số **MTTR** (Mean Time to Repair).
- **Bảo mật:** phát hiện đăng nhập bất thường, Brute Force, dữ liệu cho trung tâm điều hành an ninh (SOC).
- **Tuân thủ:** PCI DSS, ISO 27001, GDPR, Nghị định 13/2023/NĐ-CP đều yêu cầu lưu vết log.



Quản lý log trong Kibana

Năm loại log phổ biến

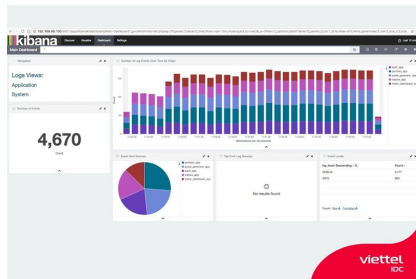
Loại log	Nội dung chính
System log	Hoạt động hệ điều hành: boot, lỗi kernel, CPU/RAM (rsyslog , systemd-journald , thư mục /var/log)
Application log	Hoạt động phần mềm, lỗi API, giao dịch database, exception trong mã nguồn
Security log	Sự kiện xác thực: đăng nhập thành công/thất bại, thay đổi quyền truy cập
Access log	Web server (Nginx, Apache): địa chỉ IP, URL, mã trạng thái HTTP (200, 404, 500)
Network log	Firewall, router, switch: phát hiện port scanning, tấn công DDoS

Chuẩn Syslog

Facility Codes phân loại nguồn phát (**kern**, **user**, **mail**, **auth**); Severity Levels từ 0 (Emergency) đến 7 (Debug).

Log Server hoạt động như thế nào?

- 1 **Collection:** agent (Fluentd, Filebeat, Logstash) thu thập log từ server/application/network.
- 2 **Parsing:** chuẩn hóa dữ liệu, tách field như timestamp, severity level.
- 3 **Indexing & storage:** lưu trong storage backend để tìm kiếm nhanh trên dữ liệu lớn.
- 4 **Visualization:** dashboard hiển thị graph/chart theo thời gian thực.

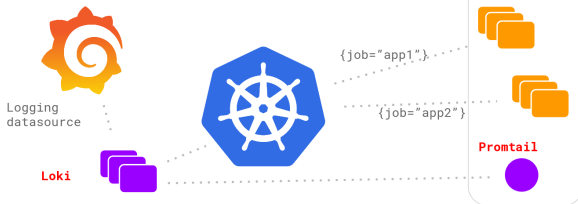


Dashboard hiển thị log trên Kibana

Quản lý log tập trung: bài toán mở rộng quy mô

- Quản lý log tập trung (CLM) thu thập toàn bộ log từ nhiều nguồn phân tán (máy chủ, ứng dụng, cloud) về một nền tảng duy nhất.
- Với hệ thống lớn có hàng trăm microservices, truy cập từng server để kiểm tra log thủ công là **bất khả thi**.
- Theo Gartner: 80% doanh nghiệp sử dụng CLM giảm được 30% thời gian xử lý sự cố.

Logging architecture



Kiến trúc log

So sánh các giải pháp quản lý log tập trung

Giải pháp	Đặc điểm nổi bật
ELK Stack	Elasticsearch + Logstash + Kibana; mạnh, linh hoạt, cộng đồng lớn – nhưng tốn RAM và vận hành phức tạp
Splunk	Thực quan hóa và bảo mật xuất sắc – nhưng chi phí bản quyền rất cao, thường chỉ hợp tập đoàn lớn
Grafana Loki	“Prometheus cho logs”; chỉ lập chỉ mục label , không index nội dung log → giảm 10–50 lần chi phí lưu trữ/CPU
digiLogs	Giải pháp Việt (TPISoftware); hiệu suất cao (> 10.000 sự kiện/giây), phù hợp quy định địa phương

Vì sao bài lab chọn Loki?

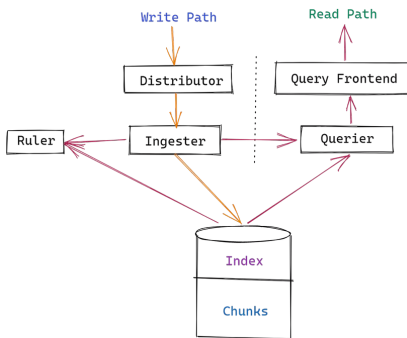
Nhẹ, tiết kiệm tài nguyên, đủ mạnh cho hệ thống WordPress + MySQL quy mô lab – không cần sức mạnh (và chi phí vận hành) của ELK hay Splunk.

Quy trình xử lý log tập trung: 5 giai đoạn

- 1 Thu thập (Collection):** agent nhẹ (Fluentd, Filebeat, Promtail) “nhặt” log từ file local hoặc stream.
- 2 Chuẩn hóa/parsing (Normalization):** tách các trường (timestamp, mức lỗi, IP nguồn) từ dữ liệu không đồng nhất.
- 3 Lưu trữ và indexing:** nén và đánh chỉ mục; hệ thống hiện đại (Loki) thường đẩy dữ liệu ra Object Storage (S3, MinIO).
- 4 Phân tích và trực quan hóa:** công cụ như Grafana tạo dashboard, vẽ biểu đồ xu hướng lỗi theo thời gian thực.
- 5 Truy vấn (Querying):** tìm kiếm nhanh trên khối lượng dữ liệu khổng lồ, ví dụ LogQL của Loki lọc theo pod/namespace.

Kiến trúc Grafana Loki: 4 thành phần cốt lõi

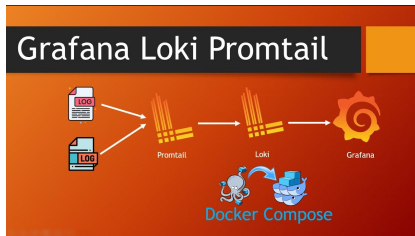
- **Distributor:** tiếp nhận log từ agent (Promtail), xác thực và chia nhỏ dữ liệu gửi đến Ingestor.
- **Ingestor:** ghi dữ liệu vào storage dài hạn, giữ log trong bộ nhớ đệm phục vụ truy vấn tức thời.
- **Querier:** xử lý truy vấn, tìm trong cả bộ nhớ đệm của Ingestor và storage dài hạn.
- **Query Frontend:** (tùy chọn) tăng tốc truy vấn bằng cách chia nhỏ và song song hóa yêu cầu lớn.



Các thành phần của Grafana Loki

Quy trình triển khai Loki bằng Docker

- 1 Chuẩn bị: cài Docker và Docker Compose trên server (Ubuntu/Debian).
- 2 Cấu hình Loki (`loki-config.yaml`): port mặc định 3100, nơi lưu trữ, chính sách retention.
- 3 Triển khai Promtail: agent trên từng host, khai báo URL Loki và đường dẫn log cần theo dõi.
- 4 Kết nối Grafana: Data Sources → Add “Loki” → nhập URL (`http://loki:3100`).



Các thành phần trong quản lý log

`docker-compose.yml`: bổ sung dịch vụ Loki

Bổ sung vào file `docker-compose.yml` đang có sẵn từ bài lab số 7 (WordPress, MySQL, Prometheus, Grafana, cAdvisor...).

```
loki:
  image: grafana/loki:latest
  container_name: loki
  restart: unless-stopped
  ports:
    - "3100:3100"
  volumes:
    - ./loki-config.yml:/etc/loki/loki-config.yml
    - loki_data:/loki
  command: -config.file=/etc/loki/loki-config.yml
  networks:
    - monitoring
```

Volume `loki_data` lưu log lâu dài; cổng 3100 dùng để Promtail/Grafana kết nối tới Loki.

`docker-compose.yml`: bổ sung dịch vụ Promtail

Promtail là agent thu thập log, chạy song song với Loki trên cùng `monitoring` network.

```
promtail:
  image: grafana/promtail:latest
  container_name: promtail
  restart: unless-stopped
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - ./promtail-config.yml:/etc/promtail/config.yml
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
  command: -config.file=/etc/promtail/config.yml
  networks:
    - monitoring
  depends_on:
    - loki
```

Mount `/var/run/docker.sock` (chỉ đọc) giúp Promtail tự động phát hiện toàn bộ container đang chạy.

Cấu hình Loki: `loki-config.yml`

```
auth_enabled: false

server:
  http_listen_port: 3100

common:
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
```

- `path_prefix` xác định thư mục lưu chunks/rules bên trong container.
- `replication_factor: 1` vì đây là một node Loki đơn (không high-availability).
- Ngoài ra, `schema_config` chọn store `boltdb-shipper` + object store `filesystem`, đánh chỉ mục theo chu kỳ 24 giờ.

Cấu hình Promtail: kết nối tới Loki

Tệp `promtail-config.yml` cấu hình agent chạy trên từng host.

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push
```

- `positions` lưu vị trí đã đọc trong từng file log, tránh đọc trùng khi restart.
- `clients` khai báo endpoint push log của Loki.

Cấu hình Promtail: tự động phát hiện container

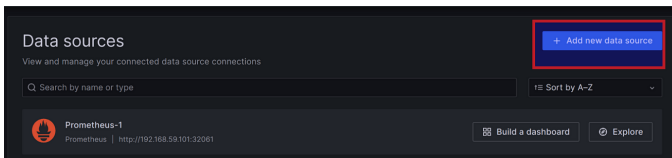
```
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 15s
    relabel_configs:
      - source_labels: ['__meta_docker_container_name']
        regex: '/(.*?)'
        target_label: container
      - source_labels:
          ↪ ['__meta_docker_container_label_com_docker_compose_service']
        target_label: service
```

Vì sao cần relabel_configs?

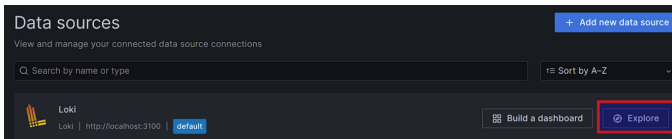
Biến metadata Docker (tên container, tên service trong compose) thành nhãn **container** và **service** – đây chính là nhãn dùng để lọc log trong LogQL.

Kết nối Loki vào Grafana làm Data Source

- Vào Grafana (~ `http://localhost:3000`) → Configuration → Data Sources → Add data source → chọn **Loki**.
- Điền: Name = `Loki`, URL = `http://loki:3100` → Save & Test.



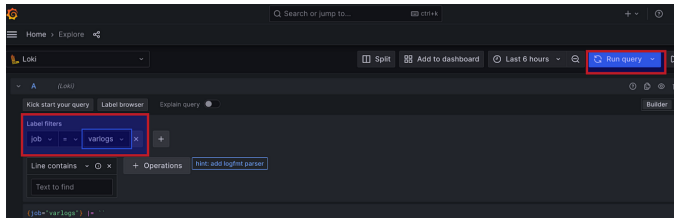
Thêm mới data source trong Grafana



Thiết lập thành công

Truy vấn log trên Grafana Explore

- Sau khi thêm data source, dùng **Explore**: chọn Loki → viết query → chuyển sang **Table** hoặc **Logs** view.
- Có thể query nhanh theo label, ví dụ `container="wordpress"` hoặc `service="mysql"`.
- Có thể tạo dashboard kết hợp Log + Metrics từ cùng một truy vấn Explore.



Thiết lập logfile và Run Query

Lưu ý khi vận hành Promtail

Lưu ý vận hành

- Dịch vụ Promtail đang dần **EOL** → khuyến nghị dùng **Grafana Alloy** cho production (một agent duy nhất cho cả metrics + logs).
- Có thể lọc log theo nhiều label khác nhau: container name, service name, environment...
- Để lưu log lâu dài, có thể chuyển sang S3/Object Storage thay vì lưu trên filesystem.

Một số truy vấn LogQL thường dùng

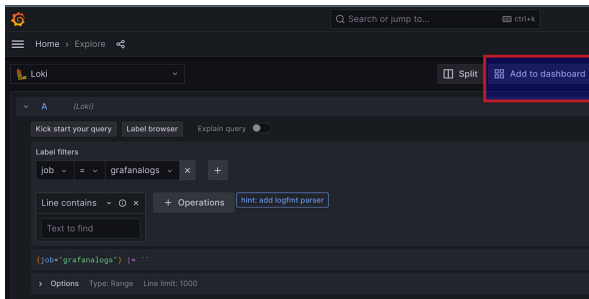
<code>{container="wordpress"}</code>	<code># Toàn bộ log WordPress</code>
<code>{container="mysql"}</code>	<code># Toàn bộ log MySQL</code>
<code>{container="mysql"} = "slow" or = "long query"</code>	<code># Query chậm</code>
<code>{container="mysql"} = "Access denied"</code>	<code># Kết nối bị từ chối</code>
<code>{container=~"wordpress mysql"} = "error"</code>	<code># Lỗi của cả 2 service</code>
<code>{service="wordpress"} or {service="mysql"}</code>	<code># Theo nhãn service</code>

Đọc một truy vấn LogQL

Toán tử `=~` là biểu thức chính quy, cho phép khớp nhiều container cùng lúc (ví dụ `wordpress|mysql`); toán tử `|=` lọc các dòng log có chứa chuỗi phía sau.

Dashboard kết hợp Log + Metrics

- **Từ Loki:** Log WordPress realtime, Top 10 request chậm, số lỗi theo thời gian (Stat + Time series).
- **Từ Prometheus:** CPU/Memory WordPress, số lượng kết nối MySQL.



Thêm Dashboard trong Loki để hiển thị log

Alertmanager: vai trò trong Prometheus stack

- Là thành phần của Prometheus stack, xử lý alert **sau khi** Prometheus phát hiện điều kiện vi phạm rule.
- **Gom nhóm (grouping)**: gộp nhiều alert liên quan thành một thông báo duy nhất.
- **Khử trùng lặp (deduplication)**: tránh gửi lặp lại cùng một cảnh báo.
- Gửi thông báo qua nhiều kênh: email, Slack, Telegram, Discord...

Luồng hoạt động

Prometheus (đánh giá alerting rules) → Alertmanager (group/dedupe) → kênh thông báo (email/Slack...).

`docker-compose.yml`: bổ sung dịch vụ Alertmanager

Tương tự Loki/Promtail, bổ sung Alertmanager vào `docker-compose.yml` đang chạy (đã có Prometheus).

```
alertmanager:
  image: prom/alertmanager:latest
  container_name: alertmanager
  restart: unless-stopped
  ports:
    - "9093:9093"
  volumes:
    - ./alertmanager.yml:/etc/alertmanager/config.yml
    - alertmanager_data:/alertmanager
  command:
    - '--config.file=/etc/alertmanager/config.yml'
    - '--storage.path=/alertmanager'
  networks:
    - monitoring
  depends_on:
    - prometheus
```

alertmanager.yml: route và receiver

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'default'

receivers:
- name: 'default'
  email_configs:
    - to: 'your-email@gmail.com'
```

- **group_wait**: thời gian chờ trước khi gửi thông báo **đầu tiên** của một nhóm.
- **group_interval** / **repeat_interval**: chờ trước khi gửi cảnh báo mới cho nhóm / gửi lại nếu alert vẫn còn.

`alertmanager.yml`: `inhibit_rules` – tránh cảnh báo trùng lặp

```
inhibit_rules:  
  - source_match:  
      severity: 'critical'  
    target_match:  
      severity: 'warning'  
      equal: ['alertname']
```

- Nếu đã có alert mức `critical` cho một `alertname`, Alertmanager sẽ **tự ẩn** các alert mức `warning` có cùng `alertname`.
- Giúp giảm số thông báo trùng lặp khi một sự cố gây ra cảnh báo ở nhiều mức nghiêm trọng cùng lúc.

`prometheus.yml`: kết nối tới Alertmanager

Prometheus cần khai báo địa chỉ Alertmanager và thư mục chứa rule để phát hiện điều kiện cảnh báo.

```
global:
  scrape_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 'alertmanager:9093'

rule_files:
  - "/etc/prometheus/rules/*.yml"  # Thư mục chứa alerting rules

scrape_configs:
  # ... (giữ nguyên các job cũ)
```

rules/alerts.yml: phân loại mức độ nghiêm trọng

```
groups:
- name: wordpress-mysql-alerts
  rules:
    - alert: WordPressHighCPU
      expr: rate(container_cpu_usage_seconds_total{container="wordpress"}[5m]) * 100 > 80
      for: 2m
      labels:
        severity: warning

    - alert: WordPressHighMemory
      expr: container_memory_usage_bytes{container="wordpress"} /
      ↪ container_spec_memory_limit_bytes{container="wordpress"} * 100 > 85
      for: 2m
      labels:
        severity: critical
```

Câu hỏi thảo luận

Vì sao `WordPressHighMemory` được gán `critical` còn `WordPressHighCPU` chỉ là `warning`? Nên dựa trên cơ sở nào để phân loại mức độ nghiêm trọng một alert rule?

Kiểm tra hệ thống cảnh báo hoạt động đúng

- Khởi động lại: `docker compose down` rồi `docker compose up -d`.
- **Alertmanager UI**: `http://localhost:9093` – xem alert đang active, đã gom nhóm ra sao.
- **Prometheus UI** (`http://localhost:9090`) → mục **Alerts**: kiểm tra rule đang bật (active).
- **Grafana**: có thể tạo alerting rule trực tiếp từ Prometheus datasource.

Tổng kết chương

Những điểm cần ghi nhớ

- Log là “hộp đen” của hệ thống; quản lý log tập trung giúp giám sát, xử lý sự cố, bảo mật và tuân thủ hiệu quả hơn kiểm tra thủ công.
- Quy trình xử lý log tập trung gồm 5 giai đoạn: thu thập, chuẩn hóa, lưu trữ/indexing, trực quan hóa và truy vấn.
- Grafana Loki chỉ lập chỉ mục nhãn thay vì nội dung log nên tiết kiệm tài nguyên hơn nhiều so với ELK Stack hay Splunk.
- Alertmanager gom nhóm và khử trùng lặp alert từ Prometheus trước khi gửi thông báo; kết hợp Monitoring, Logging và Alertmanager giúp xử lý sự cố nhanh hơn.